شرح سطر بسطر ٣ — منطق الملفات: رمز الطالب والرمز السري والمستودع والمزوّدات

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

هنا «عقل» نظام الملفات: أدوات صرفة (رمز الطالب، تجزئة الرمز السري والقفل، رمز الصف، السلسلة)، ثم المستودع `ProfileRepository` الذي يجمعها مع قاعدة البيانات، ثم مزوّدات Riverpod التي توصله للشاشات.

ما يميّز هذه الأدوات أنها دوال صرفة بلا اعتماد على Flutter، فتُختبر بسهولة وبساعة وهمية.

تنبيه أمني مهم: الرمز السري هنا قفل خفيف بين الإخوة على هاتف مشترك، وليس حماية أمنية قوية.

lib/core/utils/student_code.dart — رمز الطالب RZ-XXXX

lib/core/utils/student_code.dart

السطر 1

```
1   import 'dart:math';
```

ماذا يفعل؟ استيراد `dart:math`.

لماذا بهذه الطريقة؟ للأعداد العشوائية (`Random`).

الأسطر 3–5

```
3   /// رمز الطالب بصيغة `RZ-XXXX`. 0 لتجنّب الالتباس بالأحرف O/1/I/L.
4   /// التحقق من التفرّد على مستوى كل الأجهزة يأتي مع المزامنة لاحقًا.
5   abstract final class StudentCode {
```

ماذا يفعل؟ تعليق وصنف (`abstract final` `StudentCode`).

لماذا بهذه الطريقة؟ الأحرف بلا O/1/I/L/0 لأنها تلتبس عند القراءة أو الكتابة بخط اليد. التفرّد على كل الأجهزة يأتي مع المزامنة.

السطر 6

```
6     static const prefix = 'RZ-';
```

ماذا يفعل؟ البادئة `RZ-`.

لماذا بهذه الطريقة؟ تميّز رمز الطالب عن رمز الصف.

الأسطر 8–9

```
8     /// طول الجزء العشوائي. غيّره من هنا فقط.
9     static const randomLength = 4;
```

ماذا يفعل؟ طول الجزء العشوائي = 4، ثابت واحد.

لماذا بهذه الطريقة؟ لتغيير الطول لاحقًا نعدّل سطرًا واحدًا فقط.

السطر 11

```
11    static const alphabet = '23456789ABCDEFGHJKMNPQRSTUVWXYZ';
```

ماذا يفعل؟ الأحرف المسموحة: الأرقام 2–9 والحروف الكبيرة بلا I وا وO.

لماذا بهذه الطريقة؟ أحرف واضحة تقلّل أخطاء النسخ.

الأسطر 13–20

```
13    static String generate([Random? random]) {
14      final rng = random ?? Random.secure();
15      final body = List.generate(
16        randomLength,
17        (_) => alphabet[rng.nextInt(alphabet.length)],
18      ).join();
19      return '$prefix$body';
20    }
```

ماذا يفعل؟ `generate`: يختار 4 أحرف عشوائية من الأبجدية ويلصقها بالبادئة.

لماذا بهذه الطريقة؟ `Random.secure()` افتراضيًا (عشوائية قوية)، وقابل للاستبدال بـ `Random` ببذرة ثابتة في الاختبارات.

الأسطر 22–25

```
22    /// يوحّد الإدخال: أحرف كبيرة بلا مسافات.
23    static String normalize(String input) {
24      return input.replaceAll(RegExp(r'\s+'), '').toUpperCase();
25    }
```

ماذا يفعل؟ `normalize`: يحذف المسافات ويحوّل لأحرف كبيرة.

لماذا بهذه الطريقة؟ الطالب قد يكتب الرمز صغيرًا أو بمسافات.

الأسطر 27–32

```
27    static bool isValid(String code) {
28      final pattern = RegExp(
29        '^${RegExp.escape(prefix)}[$alphabet]{$randomLength}\$',
30      );
31      return pattern.hasMatch(code);
32    }
```

ماذا يفعل؟ `isValid`: تعبير نمطي يتحقق من البادئة وطول الجزء العشوائي وأنه من الأبجدية المسموحة.

لماذا بهذه الطريقة؟ `RegExp.escape(prefix)` يهرّب الشرطة. `\$` لإنهاء النمط.

الأسطر 34–40

```
  /// يولّد رمزًا غير مستخدم محليًا حسب [exists]. يرمي خطأ إن تعذّر ذلك
  /// بعد [maxTries] (احتمال ضئيل جدًا) محاولة.
  static Future<String> generateUnique(
    Future<bool> Function(String code) exists, {
    Random? random,
    int maxTries = 50,
  }) async {
```

ماذا يفعل؟ `generateUnique`: يقبل دالة `exists` للتحقق من الاستعمال، وعدد محاولات أقصى 50.

لماذا بهذه الطريقة؟ الدالة تُحقن من الخارج فلا يعتمد الملف على قاعدة البيانات.

الأسطر 41–44

```
    for (var i = 0; i < maxTries; i++) {
      final code = generate(random);
      if (!await exists(code)) return code;
    }
```

ماذا يفعل؟ حلقة تولّد رمزًا وتتحقق: إن لم يوجد نعيده.

لماذا بهذه الطريقة؟ احتمال التكرار ضئيل جدًا لكن نتحقق دائمًا.

الأسطر 45–47

```
    throw StateError('تعذّر توليد رمز طالب غير مكرر');
  }
}
```

ماذا يفعل؟ رمي `StateError` بعد استنفاد المحاولات ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ بدل حلقة لا نهائية.

lib/core/utils/pin_hasher.dart — تجزئة الرمز السري والقفل

lib/core/utils/pin_hasher.dart

الأسطر 1–4

```
import 'dart:convert';
import 'dart:math';

import 'package:crypto/crypto.dart';
```

ماذا يفعل؟ استيراد `dart:convert` و`dart:math` وحزمة `crypto`.

لماذا بهذه الطريقة؟ `crypto` تعطي SHA-256.

الأسطر 6–10

```
/// تجزئة الرمز السري (4 أرقام) بـ SHA-256 مع salt عشوائي لكل ملف.
///
/// تنبيه: هذا قفل خفيف بين الإخوة على الهاتف المشترك، وليس حماية أمنية قوية.
/// فضاء الرموز 10,000 احتمال فقط، فمن يملك قاعدة البيانات يستطيع تخمينه.
abstract final class PinHasher {
```

ماذا يفعل؟ تعليق وصف `PinHasher`.

لماذا بهذه الطريقة؟ التنبيه الأمني: فضاء الرموز 10,000 احتمال فقط، فمن يملك ملف القاعدة يستطيع تخمينه.

السطر 11

```
  static const pinLength = 4;
```

ماذا يفعل؟ طول الرمز = 4.

لماذا بهذه الطريقة؟ ثابت واحد يستعمله الإدخال والتحقق.

الأسطر 13–15

```
  static bool isValidFormat(String pin) {
    return RegExp('^\\d{$pinLength}\$').hasMatch(pin);
  }
```

ماذا يفعل؟ `isValidFormat`: أربعة أرقام بالضبط.

لماذا بهذه الطريقة؟ `\d{4}` تعبير نمطي.

الأسطر 17–21

```
  static String generateSalt([Random? random]) {
    final rng = random ?? Random.secure();
    final bytes = List<int>.generate(16, (_) => rng.nextInt(256));
    return base64UrlEncode(bytes);
  }
```

ماذا يفعل؟ `generateSalt`: 16 بايت عشوائية بترميز base64url.

لماذا بهذه الطريقة؟ الـ salt يجعل التجزئة مختلفة لكل ملف حتى لو تطابق الرمزان، ويفشل هجوم جداول التجزئة الجاهزة.

الأسطر 23–25

```
  static String hash(String pin, String salt) {
    return sha256.convert(utf8.encode('$salt:$pin')).toString();
  }
```

ماذا يفعل؟ `hash`: SHA-256 للنص `salt:pin`.

لماذا بهذه الطريقة؟ ندمج الـ salt والرمز قبل التجزئة. الناتج نص سداسي عشري.

الأسطر 27–29

```
27    static bool verify(String pin, {required String hash, required String salt}) {
28      final candidate = PinHasher.hash(pin, salt);
29      if (candidate.length != hash.length) return false;
```

ماذا يفعل؟ `verify`: نجزّئ الرمز المُدخل بنفس الـ salt؛ إن اختلف الطول نرفض.

لماذا بهذه الطريقة؟ نقارن بالمخزّن.

الأسطر 30–36

```
30      // مقارنة بزمن ثابت.
31      var diff = 0;
32      for (var i = 0; i < candidate.length; i++) {
33        diff |= candidate.codeUnitAt(i) ^ hash.codeUnitAt(i);
34      }
35      return diff == 0;
36    }
```

ماذا يفعل؟ مقارنة بزمن ثابت: نجمع فروق كل الأحرف بـ XOR بدل التوقف عند أول اختلاف.

لماذا بهذه الطريقة؟ تمنع هجوم القياس الزمني. مبالغة لتهديد ضعيف لكنها عادة سليمة.

السطر 37

```
37  }
```

ماذا يفعل؟ إغلاق الصنف.

لماذا بهذه الطريقة؟ نهاية `PinHasher`.

الأسطر 39–41

```
39  /// محاولات خاطئة متتالية [maxAttempts] بعد [lockDuration] يوقف إدخال الرمز.
40  /// الحالة في الذاكرة فقط (تُصفَّر بإعادة تشغيل التطبيق)، كافية لقفل خفيف.
41  class PinLockout {
```

ماذا يفعل؟ تعليق وصنف `PinLockout`.

لماذا بهذه الطريقة؟ الحالة في الذاكرة فقط وتُصفَّر بإعادة تشغيل التطبيق: كافية لقفل خفيف.

الأسطر 42–46

```
42    PinLockout({
43      this.maxAttempts = 5,
44      this.lockDuration = const Duration(seconds: 30),
45      DateTime Function()? now,
46    }) : _now = now ?? DateTime.now;
```

ماذا يفعل؟ الباني: 5 محاولات و30 ثانية افتراضيًا ودالة `now` اختيارية.

لماذا بهذه الطريقة؟ حقن الساعة يسمح باختبار القفل بزمن وهمي دون انتظار 30 ثانية.

الأسطر 48–53

```
  final int maxAttempts;
  final Duration lockDuration;
  final DateTime Function() _now;

  int _failures = 0;
  DateTime? _lockedUntil;
```

ماذا يفعل؟ الحقول: الإعدادات والساعة وعدّاد الفشل ووقت انتهاء القفل.

لماذا بهذه الطريقة؟ العدّاد وموعد الانتهاء خاصان.

الأسطر 55–66

```
  /// الزمن المتبقي للقفل، أو null إن لم يكن مقفلًا.
  Duration? get remaining {
    final until = _lockedUntil;
    if (until == null) return null;
    final left = until.difference(_now());
    if (left <= Duration.zero) {
      _lockedUntil = null;
      _failures = 0;
      return null;
    }
    return left;
  }
```

ماذا يفعل؟ `remaining`: الزمن المتبقي للقفل أو `null`. إن انتهى الوقت نصفّر القفل والعدّاد.

لماذا بهذه الطريقة؟ القراءة نفسها تنظّف الحالة المنتهية، فلا نحتاج مؤقّتًا.

الأسطر 68–70

```
  bool get isLocked => remaining != null;

  int get failures => _failures;
```

ماذا يفعل؟ `isLocked` و`failures`.

لماذا بهذه الطريقة؟ أدوات قراءة سريعة.

الأسطر 72–75

```
  void registerSuccess() {
    _failures = 0;
    _lockedUntil = null;
  }
```

ماذا يفعل؟ `registerSuccess`: يصفّر العدّاد والقفل.

لماذا بهذه الطريقة؟ الدخول الناجح يعيد البداية.

الأسطر 77–83

```
  void registerFailure() {
    _failures++;
    if (_failures >= maxAttempts) {
      _lockedUntil = _now().add(lockDuration);
    }
  }
}
```

ماذا يفعل؟ `registerFailure`: يزيد العدّاد؛ عند بلوغ الحد يضبط موعد انتهاء القفل بعد المدة. ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ القفل يبدأ عند المحاولة الخامسة.

lib/core/utils/class_code.dart — رمز الصف

lib/core/utils/class_code.dart

الأسطر 1–3

```
/// رمز الصف الاختياري الذي يعطيه المعلم، مثل `GZ-5A`.
/// يُخزَّن محليًا فقط الآن؛ الانضمام الفعلي لمجموعة يأتي مع المزامنة.
abstract final class ClassCode {
```

ماذا يفعل؟ تعليق وصنف `ClassCode`.

لماذا بهذه الطريقة؟ رمز اختياري يعطيه المعلم (مثل `GZ-5A`). يُخزَّن محليًا الآن؛ الانضمام الفعلي يأتي مع المزامنة.

السطر 4

```
  static const maxLength = 16;
```

ماذا يفعل؟ أقصى طول 16.

لماذا بهذه الطريقة؟ حد معقول يمنع إدخالًا ضخمًا.

الأسطر 6–9

```
  /// أحرف كبيرة بلا مسافات.
  static String normalize(String input) {
    return input.replaceAll(RegExp(r'\s+'), '').toUpperCase();
  }
```

ماذا يفعل؟ `normalize`: أحرف كبيرة بلا مسافات.

لماذا بهذه الطريقة؟ الطالب قد يكتب `gz-5a` أو بمسافات.

الأسطر 11–15

```
11    /// حروف إنجليزية وأرقام وشرطة فقط، ويبدأ وينتهي بحرف أو رقم.
12    static bool isValid(String normalized) {
13      if (normalized.isEmpty || normalized.length > maxLength) return false;
14      return RegExp(r'^[A-Z0-9]+(-[A-Z0-9]+)*$').hasMatch(normalized);
15    }
```

ماذا يفعل؟ `isValid`: غير فارغ وطوله ≤ 16 ويطابق: مقاطع من حروف إنجليزية/أرقام يفصلها شرطة.

لماذا بهذه الطريقة؟ يقبل `GZ-5A` ويرفض `--GZ` و`GZ-` والحروف العربية.

السطر 16

```
16  }
```

ماذا يفعل؟ إغلاق الصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/core/utils/streak.dart — السلسلة (الأيام المتتالية)

lib/core/utils/streak.dart

الأسطر 1–4

```
1   /// عدد الأيام المتتالية التي فيها نشاط، محسوبًا من أوقات النشاط (لا يُخزَّن).
2   ///
3   /// تنتهي السلسلة اليوم إن وُجد نشاط اليوم، وإلا تنتهي أمس (السلسلة ما زالت
4   /// «حيّة» حتى نهاية اليوم). إن لم يكن هناك نشاط اليوم ولا أمس فالنتيجة 0.
```

ماذا يفعل؟ تعليق شرح القاعدة.

لماذا بهذه الطريقة؟ تُحسب ولا تُخزَّن. تنتهي اليوم إن وُجد نشاط اليوم وإلا أمس (ما زالت «حيّة» حتى نهاية اليوم)، وإلا 0.

السطر 5

```
5   int currentStreak(Iterable<DateTime> activityTimes, DateTime now) {
```

ماذا يفعل؟ الدالة `currentStreak(activityTimes, now)`.

لماذا بهذه الطريقة؟ تأخذ أوقات النشاط والوقت الحالي (يُحقن للاختبار).

السطر 6

```
6     DateTime dayOf(DateTime t) => DateTime(t.year, t.month, t.day);
```

ماذا يفعل؟ دالة محلية `dayOf` تحذف الوقت وتُبقي اليوم فقط.

لماذا بهذه الطريقة؟ نقارن بالأيام لا بالساعات.

السطر 8

```
8     final days = activityTimes.map(dayOf).toSet();
```

ماذا يفعل؟ نحوّل كل الأوقات إلى مجموعة أيام فريدة.

لماذا بهذه الطريقة؟ عدة أنشطة في يوم واحد تُحسب يومًا واحدًا.

الأسطر 9–13

```
9     var cursor = dayOf(now);
10    if (!days.contains(cursor)) {
11      cursor = cursor.subtract(const Duration(days: 1));
12      cursor = DateTime(cursor.year, cursor.month, cursor.day);
13    }
```

ماذا يفعل؟ نبدأ من اليوم؛ إن لم يكن فيه نشاط نبدأ من أمس.

لماذا بهذه الطريقة؟ هذا ما يُبقي السلسلة حيّة طوال اليوم الحالي.

الأسطر 15–21

```
15    var streak = 0;
16    while (days.contains(cursor)) {
17      streak++;
18      final previous = cursor.subtract(const Duration(days: 1));
19      cursor = DateTime(previous.year, previous.month, previous.day);
20    }
21    return streak;
```

ماذا يفعل؟ حلقة تعدّ الأيام المتتالية رجوعًا ما دام اليوم موجودًا في المجموعة ثم تعيد العدد.

لماذا بهذه الطريقة؟ نعيد بناء `DateTime` بالسنة/الشهر/اليوم لتجنّب مشاكل الصيف/الشتاء.

السطر 22

```
22  }
```

ماذا يفعل؟ إغلاق الدالة.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/features/profiles/profile_repository.dart — مستودع الملفات

`lib/features/profiles/profile_repository.dart`

الأسطر 1–4

```
1   import 'dart:math';
2
3   import 'package:drift/drift.dart';
4   import 'package:uuid/uuid.dart';
```

ماذا يفعل؟ استيراد `dart:math` وDrift و`uuid`.

لماذا بهذه الطريقة؟ `Value` من Drift للأعمدة الاختيارية، و`Uuid` لمعرّفات الملفات.

الأسطر 6–10

```dart
import '../../core/db/app_database.dart';
import '../../core/db/daos/settings_dao.dart';
import '../../core/utils/class_code.dart';
import '../../core/utils/pin_hasher.dart';
import '../../core/utils/student_code.dart';
```

ماذا يفعل؟ استيراد القاعدة وDAO الإعدادات وأدوات رمز الصف والرمز السري ورمز الطالب.

لماذا بهذه الطريقة؟ المستودع يجمعها.

السطر 12

```dart
enum PinCheck { ok, wrong, locked }
```

ماذا يفعل؟ تعداد `PinCheck` : ناجح / خاطئ / مقفل.

لماذا بهذه الطريقة؟ نتيجة التحقق ثلاثية الحالة.

الأسطر 14–23

```dart
class PinVerification {
  const PinVerification(this.check, {this.lockedFor});

  final PinCheck check;

  /// الزمن المتبقي للقفل عند `locked`.
  final Duration? lockedFor;

  bool get isOk => check == PinCheck.ok;
}
```

ماذا يفعل؟ صنف `PinVerification` : النتيجة ومعها الزمن المتبقي عند القفل، وخاصية `isOk` .

لماذا بهذه الطريقة؟ يمرّر للشاشة ما تحتاجه لعرض الرسالة والعدّ التنازلي.

الأسطر 25–26

```dart
/// ملفات الطلاب على الجهاز: الإنشاء، الملف النشط، والرمز السري.
class ProfileRepository {
```

ماذا يفعل؟ تعليق وصنف `ProfileRepository` .

لماذا بهذه الطريقة؟ مكان قواعد العمل الخاصة بالملفات.

الأسطر 27–34

```
27    ProfileRepository(
28      this._db, {
29      DateTime Function()? now,
30      Random? random,
31      PinLockout Function()? lockoutFactory,
32    }) : _now = now ?? DateTime.now,
33         _random = random,
34         _lockoutFactory = lockoutFactory ?? (() => PinLockout(now: now));
```

ماذا يفعل؟ الباني: القاعدة مطلوبة، وساعة وعشوائية ومصنع قفل اختيارية.

لماذا بهذه الطريقة؟ كلها قابلة للحقن في الاختبارات. إن لم يُمرَّر مصنع القفل نصنع `PinLockout` بنفس الساعة.

الأسطر 36–41

```
36    final AppDatabase _db;
37    final DateTime Function() _now;
38    final Random? _random;
39    final PinLockout Function() _lockoutFactory;
40    final _uuid = const Uuid();
41    final Map<String, PinLockout> _lockouts = {};
```

ماذا يفعل؟ الحقول: القاعدة والساعة والعشوائية والمصنع و`Uuid` وخريطة الأقفال.

لماذا بهذه الطريقة؟ خريطة `_lockouts` تربط كل ملف بقفله الخاص (المفتاح معرّف الملف).

الأسطر 43–51

```
43    Stream<List<Profile>> watchAll() => _db.profilesDao.watchAll();
44
45    Future<List<Profile>> getAll() => _db.profilesDao.getAll();
46
47    Future<Profile?> getById(String id) => _db.profilesDao.getById(id);
48
49    Stream<Profile?> watchById(String id) => _db.profilesDao.watchById(id);
50
51    Future<int> count() => _db.profilesDao.count();
```

ماذا يفعل؟ دوال القراءة: `watchAll` و`getAll` و`getById` و`watchById` و`count`.

لماذا بهذه الطريقة؟ تمرّ للـ DAO فقط.

الأسطر 53–60

```
53    /// ينشئ ملفًا جديدًا ويجعله الملف النشط.
54    Future<Profile> create({
55      required String name,
56      required int grade,
57      required ProfileColor color,
58      String? pin,
59      String? classCode,
60    }) async {
```

ماذا يفعل؟ توقيع `create`: الاسم والصف واللون مطلوبة، والرمز السري ورمز الصف اختياريان.

لماذا بهذه الطريقة؟ نقطة الدخول الوحيدة لإنشاء ملف.

الأسطر 61–63

```
61      final cleanName = name.trim();
62      if (cleanName.isEmpty) throw ArgumentError.value(name, 'name');
63      if (grade < 1 || grade > 12) throw ArgumentError.value(grade, 'grade');
```

ماذا يفعل؟ قصّ الاسم ورفض الفارغ ورفض صف خارج 1–12.

لماذا بهذه الطريقة؟ تحقق ثانٍ بعد تحقق الواجهة (دفاع في العمق).

الأسطر 65–71

```
65      String? pinHash;
66      String? pinSalt;
67      if (pin != null) {
68        if (!PinHasher.isValidFormat(pin)) throw ArgumentError.value(pin, 'pin');
69        pinSalt = PinHasher.generateSalt(_random);
70        pinHash = PinHasher.hash(pin, pinSalt);
71      }
```

ماذا يفعل؟ إن مُرّر رمز سري: نتحقق من صيغته ثم نولّد salt ونجزّئ.

لماذا بهذه الطريقة؟ التجزئة داخل المستودع فلا تعرف الشاشة شيئًا عن التجزئة.

الأسطر 73–79

```
73      String? cleanClassCode;
74      if (classCode != null && classCode.trim().isNotEmpty) {
75        cleanClassCode = ClassCode.normalize(classCode);
76        if (!ClassCode.isValid(cleanClassCode)) {
77          throw ArgumentError.value(classCode, 'classCode');
78        }
79      }
```

ماذا يفعل؟ إن مُرّر رمز صف غير فارغ: نوحّده ونتحقق من صيغته.

لماذا بهذه الطريقة؟ رمز صف خاطئ يرمي خطأً فلا نخزّن بيانات فاسدة.

الأسطر 81–84

```
81      final code = await StudentCode.generateUnique(
82        (c) async => await _db.profilesDao.getByStudentCode(c) != null,
83        random: _random,
84      );
```

ماذا يفعل؟ نولّد رمز طالب غير مستعمل محليًا بسؤال القاعدة.

لماذا بهذه الطريقة؟ `generateUnique` تُمرَّر لها دالة تتحقق عبر الـ DAO.

الأسطر 85–86

```
85      final now = _now();
86      final id = _uuid.v4();
```

ماذا يفعل؟ نأخذ الوقت الحالي ونولّد معرّف uuid.

لماذا بهذه الطريقة؟ الوقت للإنشاء وآخر نشاط.

الأسطر 88–101

```
    await _db.profilesDao.insertProfile(
      ProfilesCompanion.insert(
        id: id,
        name: cleanName,
        grade: grade,
        color: color,
        pinHash: Value(pinHash),
        pinSalt: Value(pinSalt),
        studentCode: code,
        classCode: Value(cleanClassCode),
        createdAt: now,
        lastActiveAt: now,
      ),
    );
```

ماذا يفعل؟ إدخال الصف في القاعدة بـ `ProfilesCompanion.insert`.

لماذا بهذه الطريقة؟ `Value(...)` للأعمدة الاختيارية (`null` مسموح).

الأسطر 102–103

```
    await setActive(id);
    return (await getById(id))!;
```

ماذا يفعل؟ نجعل الملف الجديد نشطًا ثم نعيد قراءته.

لماذا بهذه الطريقة؟ ليبدأ الطالب بملفه فورًا.

السطر 104

```
  }
```

ماذا يفعل؟ إغلاق `create`.

لماذا بهذه الطريقة؟ نهاية الدالة.

الأسطر 106–110

```
  // الملف النشط
  Future<void> setActive(String id) async {
    await _db.profilesDao.touch(id, _now());
    await _db.settingsDao.write(SettingsDao.activeProfileKey, id);
  }
```

ماذا يفعل؟ `setActive`: يحدّث `lastActiveAt` ويحفظ المعرّف في الإعدادات.

لماذا بهذه الطريقة؟ يجعل الملف الأحدث نشاطًا والأخير المحفوظ.

الأسطر 112–117

```
  /// معرّف الملف النشط الأخير، أو null إن لم يوجد أو حُذف ملفه.
  Future<String?> readActiveId() async {
    final id = await _db.settingsDao.read(SettingsDao.activeProfileKey);
    if (id == null) return null;
    return await getById(id) == null ? null : id;
  }
```

ماذا يفعل؟ `readActiveId`: يقرأ الملف النشط المحفوظ ويتحقق أنه ما زال موجودًا.

لماذا بهذه الطريقة؟ إن حُذف ملفه نعيد `null` بدل معرّف يتيم.

الأسطر 119–122

```
  // الرمز السري
  Future<PinVerification> verifyPin(String profileId, String pin) async {
    final profile = await getById(profileId);
    if (profile == null) throw StateError('ملف غير موجود');
```

ماذا يفعل؟ `verifyPin` يبدأ بقراءة الملف ورمي خطأ إن لم يوجد.

لماذا بهذه الطريقة؟ لا معنى للتحقق من ملف غير موجود.

الأسطر 123–127

```
    final hash = profile.pinHash;
    final salt = profile.pinSalt;
    if (hash == null || salt == null) {
      return const PinVerification(PinCheck.ok);
    }
```

ماذا يفعل؟ نقرأ التجزئة والـ salt؛ إن كان أحدهما `null` فلا رمز سري وننجح.

لماذا بهذه الطريقة؟ ملف بلا رمز يقبل أي إدخال.

الأسطر 129–133

```
    final lockout = _lockouts.putIfAbsent(profileId, _lockoutFactory);
    final locked = lockout.remaining;
    if (locked != null) {
      return PinVerification(PinCheck.locked, lockedFor: locked);
    }
```

ماذا يفعل؟ نجلب قفل الملف (ننشئه عند أول مرة)؛ إن كان مقفلًا نعيد `locked` مع الزمن المتبقي.

لماذا بهذه الطريقة؟ أثناء القفل يُرفض حتى الرمز الصحيح.

الأسطر 135–138

```
    if (PinHasher.verify(pin, hash: hash, salt: salt)) {
      lockout.registerSuccess();
      return const PinVerification(PinCheck.ok);
    }
```

ماذا يفعل؟ إن طابق الرمز: نصفّر القفل ونعيد `ok`.

لماذا بهذه الطريقة؟ نجاح الدخول.

الأسطر 139–144

```
    lockout.registerFailure();
    final nowLocked = lockout.remaining;
    return nowLocked == null
        ? const PinVerification(PinCheck.wrong)
        : PinVerification(PinCheck.locked, lockedFor: nowLocked);
  }
```

ماذا يفعل؟ وإلا نسجّل فشلًا؛ إن صار مقفلًا نعيد `locked`، وإلا `wrong`.

لماذا بهذه الطريقة؟ المحاولة الخامسة تُرجع `locked` مباشرة.

الأسطر 146–155

```
  Future<void> setPin(String profileId, String pin) async {
    if (!PinHasher.isValidFormat(pin)) throw ArgumentError.value(pin, 'pin');
    final salt = PinHasher.generateSalt(_random);
    await _db.profilesDao.setPin(
      profileId,
      hash: PinHasher.hash(pin, salt),
      salt: salt,
    );
    _lockouts.remove(profileId);
  }
```

ماذا يفعل؟ `setPin`: نتحقق من الصيغة ونولّد salt جديدًا ونحفظ التجزئة ونصفّر القفل.

لماذا بهذه الطريقة؟ تغيير الرمز يعيد البداية.

الأسطر 157–160

```
  Future<void> clearPin(String profileId) async {
    await _db.profilesDao.clearPin(profileId);
    _lockouts.remove(profileId);
  }
```

ماذا يفعل؟ `clearPin`: يلغي الرمز ويصفّر القفل.

لماذا بهذه الطريقة؟ لأن الملف لم يعد محميًا.

الأسطر 162–167

```
  /// هل [studentCode] هو رمز طالب هذا الملف؟ (يوحّد الإدخال قبل المقارنة.)
  Future<bool> matchesStudentCode(String profileId, String studentCode) async {
    final profile = await getById(profileId);
    if (profile == null) return false;
    return StudentCode.normalize(studentCode) == profile.studentCode;
  }
```

ماذا يفعل؟ `matchesStudentCode`: هل الإدخال هو رمز الطالب لهذا الملف؟

لماذا بهذه الطريقة؟ تتحقق فقط ولا تغيّر شيئًا؛ تستعملها واجهة «نسيت الرمز؟» قبل عرض الخيارين.

الأسطر 169–175

```
  /// «إن طابق»: نسيت الرمز؟ [studentCode] رمز هذا الملف يُعيَّن [newPin]
  /// ويُرجع true. (أو null إن كان الرمز السري يُلغى)
  Future<bool> resetPinWithStudentCode(
    String profileId,
    String studentCode, {
    String? newPin,
  }) async {
```

ماذا يفعل؟ توقيع `resetPinWithStudentCode`: ملف ورمز طالب ورمز جديد اختياري.

لماذا بهذه الطريقة؟ رمز الطالب هو «مفتاح الاستعادة».

الأسطر 176–180

```
    final profile = await getById(profileId);
    if (profile == null) return false;
    if (StudentCode.normalize(studentCode) != profile.studentCode) {
      return false;
    }
```

ماذا يفعل؟ نرفض (false) إن لم يوجد الملف أو لم يطابق الرمز.

لماذا بهذه الطريقة؟ لا نغيّر شيئًا بلا إثبات.

الأسطر 181–187

```
    if (newPin == null) {
      await clearPin(profileId);
    } else {
      await setPin(profileId, newPin);
    }
    return true;
  }
```

ماذا يفعل؟ إن لم يُمرَّر رمز جديد نلغي الرمز السري، وإلا نعيّنه ونعيد `true`.

لماذا بهذه الطريقة؟ خيارا الواجهة: «إلغاء» أو «تعيين رمز جديد».

الأسطر 189–205

```
189   /// يحذف الملف نهائيًا مع كل ما يخصّه: التقدّم والمحاولات والإتقان وتفضيلاته.
190   /// إن كان هو الملف النشط يُمسح مؤشّر الملف النشط أيضًا. يعيد عدد الملفات
191   /// المتبقية. لا رجوع عن هذا (لا مزامنة بعد).
192   Future<int> delete(String profileId) async {
193     await _db.transaction(() async {
194       await _db.activityDao.clearActivity(profileId);
195       await _db.settingsDao.removeByPrefix(
196         SettingsDao.profilePrefsPrefix(profileId),
197       );
198       if (await readActiveIdRaw() == profileId) {
199         await _db.settingsDao.remove(SettingsDao.activeProfileKey);
200       }
201       await _db.profilesDao.deleteById(profileId);
202     });
203     _lockouts.remove(profileId);
204     return count();
205   }
```

ماذا يفعل؟ `delete(profileId)`: معاملة واحدة تمسح نشاط الملف، ثم تفضيلاته، ثم مؤشّر الملف النشط إن كان هو، ثم الملف نفسه؛ وبعدها نصفّر عدّاد قفل الرمز السري ونعيد عدد الملفات المتبقية.

لماذا بهذه الطريقة؟ المعاملة (`transaction`) تجعل الحذف «كله أو لا شيء»: لو انقطع بين الخطوات لا يبقى ملف نصف محذوف. الترتيب: الأبناء أولًا ثم الأب. مؤشّر الملف النشط يُمسح فقط إن كان لهذا الملف، فحذف ملف آخر لا يُخرج الطالب الحالي. عدد المتبقين تقرر به الشاشة وجهتها.

الأسطر 207–209

```
207   /// معرّف الملف النشط كما هو محفوظ (دون التحقق من وجود الملف).
208   Future<String?> readActiveIdRaw() =>
209       _db.settingsDao.read(SettingsDao.activeProfileKey);
```

ماذا يفعل؟ `readActiveIdRaw`: معرّف الملف النشط المحفوظ كما هو.

لماذا بهذه الطريقة؟ على خلاف `readActiveId` لا تتحقق من وجود الملف، لأننا نحتاج المقارنة بالمعرّف نفسه أثناء الحذف (الملف قد يكون حُذف توًّا).

الأسطر 211–215

```
211   /// يغيّر صف الطالب (1–12). نتائجه ونجومه لا تتأثر.
212   Future<void> setGrade(String profileId, int grade) async {
213     if (grade < 1 || grade > 12) throw ArgumentError.value(grade, 'grade');
214     await _db.profilesDao.setGrade(profileId, grade);
215   }
```

ماذا يفعل؟ `setGrade`: يرفض صفًّا خارج 1–12 ثم يحدّث عبر الـ DAO.

لماذا بهذه الطريقة؟ التحقق في المستودع (مثل `create`) لا في الواجهة، فلا يصل صف غير صالح للقاعدة من أي مصدر. تغيير الصف لا يمسّ نتائج الطالب ولا نجومه.

الأسطر 217–228

```
  Future<void> setClassCode(String profileId, String? classCode) async {
    if (classCode == null || classCode.trim().isEmpty) {
      await _db.profilesDao.setClassCode(profileId, null);
      return;
    }
    final normalized = ClassCode.normalize(classCode);
    if (!ClassCode.isValid(normalized)) {
      throw ArgumentError.value(classCode, 'classCode');
    }
    await _db.profilesDao.setClassCode(profileId, normalized);
  }
}
```

ماذا يفعل؟ `setClassCode` : فارغ يلغي رمز الصف، وغير ذلك يُوحَّد ويُتحقَّق منه ثم يُحفظ، ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ نفس قواعد `create` .

lib/features/profiles/profile_providers.dart — مزوّدات الملفات

lib/features/profiles/profile_providers.dart

الأسطر 1–5

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../core/db/app_database.dart';
import '../../core/db/database_provider.dart';
import 'profile_repository.dart';
```

ماذا يفعل؟ استيراد Riverpod والقاعدة ومزوّدها والمستودع.

لماذا بهذه الطريقة؟ المزوّدات تربط الطبقات.

الأسطر 7–12

```
final profileRepositoryProvider = Provider<ProfileRepository>((ref) {
  return ProfileRepository(
    ref.watch(appDatabaseProvider),
    now: ref.watch(clockProvider),
  );
});
```

ماذا يفعل؟ `profileRepositoryProvider` ينشئ المستودع بالقاعدة والساعة.

لماذا بهذه الطريقة؟ `watch` على القاعدة والساعة فيتجدّد المستودع إن استُبدلا (في الاختبارات).

الأسطر 14–17

```
/// كل الملفات على الجهاز، الأحدث نشاطًا أولًا.
final profilesProvider = StreamProvider<List<Profile>>((ref) {
  return ref.watch(profileRepositoryProvider).watchAll();
});
```

ماذا يفعل؟ `profilesProvider` : بث بكل الملفات.

لماذا بهذه الطريقة؟ يعتمد عليه شاشة الاختيار.

الأسطر 19–22

```
/// مجموع نجوم كل ملف (معرّف الملف ← عدد النجوم). الملف بلا محاولات غير موجود.
final starsByProfileProvider = StreamProvider<Map<String, int>>((ref) {
  return ref.watch(appDatabaseProvider).activityDao.watchStarsByProfile();
});
```

ماذا يفعل؟ `starsByProfileProvider`: بث بنجوم كل ملف.

لماذا بهذه الطريقة؟ يستدعي الـ DAO مباشرة لأن الاستعلام تجميعي بلا قواعد عمل.

الأسطر 24–27

```
/// ملف واحد بمعرّفه (يتحدّث تلقائيًا). `null` إن لم يعد موجودًا.
final profileByIdProvider = StreamProvider.autoDispose.family<Profile?, String>(
  (ref, id) => ref.watch(profileRepositoryProvider).watchById(id),
);
```

ماذا يفعل؟ `profileByIdProvider`: ملف بمعرّفه كـ `family` و`autoDispose`.

لماذا بهذه الطريقة؟ `family` ليأخذ معرّفًا، و`autoDispose` ليتخلّص منه حين تُغلق الشاشة.

lib/features/profiles/active_profile_provider.dart — الملف النشط

lib/features/profiles/active_profile_provider.dart

الأسطر 1–4

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../core/db/app_database.dart';
import 'profile_providers.dart';
```

ماذا يفعل؟ استيراد Riverpod والقاعدة ومزوّدات الملفات.

لماذا بهذه الطريقة؟ نحتاج `Profile` والمستودع.

الأسطر 6–7

```
/// معرّف الملف النشط المحفوظ على الجهاز (null = لا يوجد).
class ActiveProfileIdNotifier extends AsyncNotifier<String?> {
```

ماذا يفعل؟ صنف `ActiveProfileIdNotifier` من نوع `AsyncNotifier<String?>`.

لماذا بهذه الطريقة؟ حالة غير متزامنة لأن القراءة الأولى من القاعدة.

الأسطر 8–11

```
  @override
  Future<String?> build() {
    return ref.read(profileRepositoryProvider).readActiveId();
  }
```

ماذا يفعل؟ `build`: يقرأ الملف النشط المحفوظ.

لماذا بهذه الطريقة؟ يُنفَّذ عند أول قراءة للمزوّد.

الأسطر 13–17

```
  /// يجعل [id] الملف النشط ويحفظه.
  Future<void> select(String id) async {
    await ref.read(profileRepositoryProvider).setActive(id);
    state = AsyncData(id);
  }
```

ماذا يفعل؟ `select(id)`: يحفظ الملف نشطًا في القاعدة ثم يحدّث الحالة.

لماذا بهذه الطريقة؟ نحدّث الحالة بعد الحفظ فيرى الراوتر والشاشات الاختيار فورًا.

الأسطر 19–23

```
  /// يعيد قراءة الملف النشط من القاعدة (بعد إنشاء ملف جديد مثلًا).
  Future<void> reload() async {
    state = AsyncData(await ref.read(profileRepositoryProvider).readActiveId());
  }
}
```

ماذا يفعل؟ `reload`: يعيد القراءة من القاعدة.

لماذا بهذه الطريقة؟ بعد إنشاء ملف جديد (الذي جعله نشطًا بنفسه).

الأسطر 25–28

```
final activeProfileIdProvider =
    AsyncNotifierProvider<ActiveProfileIdNotifier, String?>(
      ActiveProfileIdNotifier.new,
    );
```

ماذا يفعل؟ `activeProfileIdProvider`: تسجيل المزوّد.

لماذا بهذه الطريقة؟ يربط الصنف بـ `AsyncNotifierProvider`.

الأسطر 30–38

```
/// الملف النشط نفسه، ويتحدّث تلقائيًا عند تغيّر بياناته.
final activeProfileProvider = StreamProvider<Profile?>((ref) async* {
  final id = await ref.watch(activeProfileIdProvider.future);
  if (id == null) {
    yield null;
    return;
  }
  yield* ref.watch(profileRepositoryProvider).watchById(id);
});
```

ماذا يفعل؟ `activeProfileProvider`: بث بالملف النشط نفسه.

لماذا بهذه الطريقة؟ ننتظر المعرّف؛ إن كان `null` نصدر `null` وننتهي، وإلا نمرّر بث `watchById` فيتحدّث الملف عند تغيّر بياناته. `yield*` يمرّر البث كاملًا.